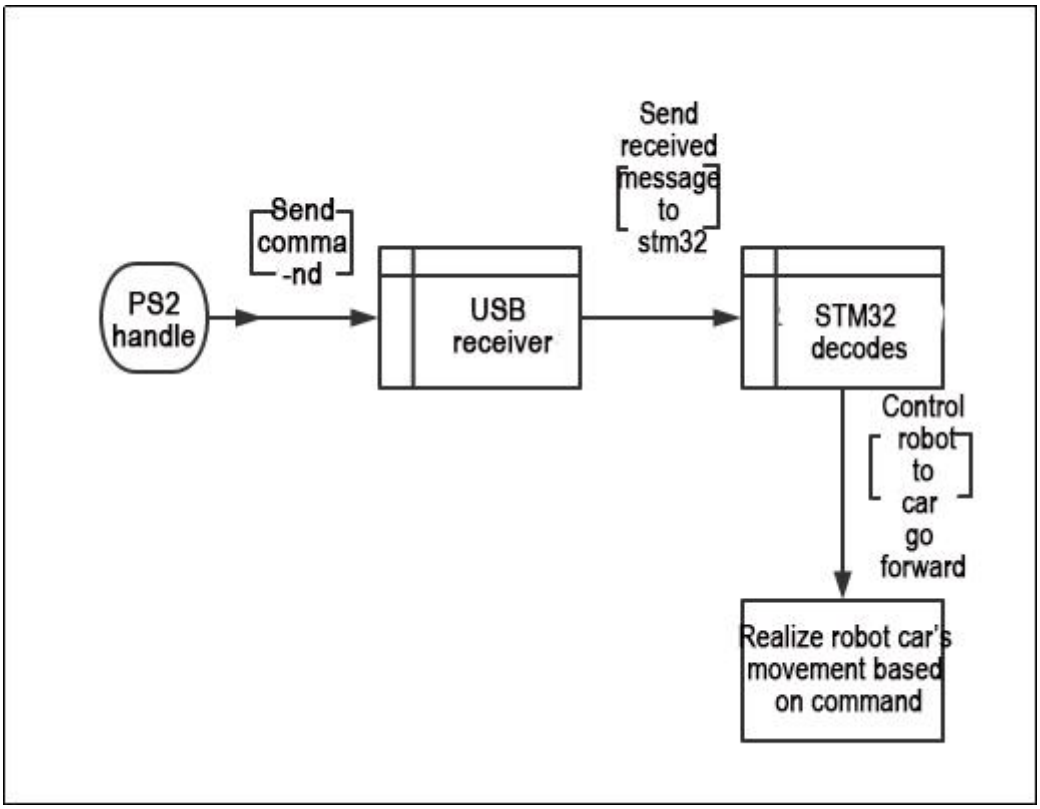
PS2 Controller USB Receiver Signal Processing Principles

In the previous course "Wireless Controller Usage," you should have learned the basic operation of the controller. In this lesson, we will learn about the working process of the PS2 controller, the principle of how the USB receiver receives the controller signal, and the working process of the STM32.

1. Program Flow

The flow process of the PS2 controller sending information to the USB receiver, and then the USB receiver sending the data to the STM32 for processing:



1. The PS2 controller sends commands wirelessly to the USB receiver.
2. The USB receiver's job is to receive the controller's commands.

3. The USB receiver first needs to establish a connection with the PS2 controller. Once connected, it transmits the received commands from the PS2 controller to the STM32.

4. The STM32 decodes the received information and executes corresponding operations based on the decoded information (e.g., making a car move forward). The car will then perform this operation.

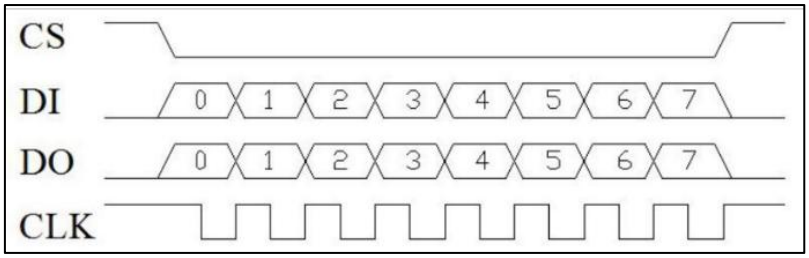
2. Source Code Path

The source code for this section is located at “Appendix/Programs/PS2 Controller Signal Processing” under “Hiwonder/System” in “gampad_handle.c” and “app.c”. You can also use Keil5 (refer to previous content for installation details) to open the project file. This can be found under “Appendix/Programs/PS2 Controller Signal Processing” in the “MDK-ARM” section on the left side of “Hiwonder/System”.

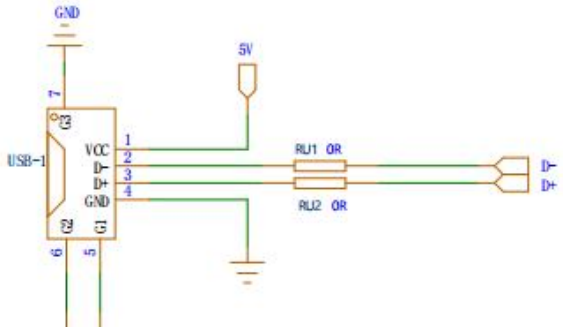
3. Working Principle Analysis

3.1 PS2 Controller Command Sending Process

The controller sends information via DI, DO (data lines): When you operate the controller, such as pressing a button, the button information is encoded into an electrical signal and sent out through the data lines.



3.2 Working Process of USB Receiver and STM32



1. The USB receiver transmits the electrical signal received from the controller over differential signals on a pair of data lines (D+ and D-), then sends it to the STM32 for decoding.

2. STM32 Decoding Operation: The STM32 reads the electrical signal on the data lines and converts these electrical signals into binary data. The STM32 decodes this data into specific commands based on the PS2 protocol rules, such as "press the X button." Then, it executes the corresponding operation, such as "move the car forward when the X button is pressed."

3.3 Knowledge Expansion

PS2 Protocol:

- Data Format: Data is sent in bytes (8 bits). Each byte includes a start bit (low level), 8 data bits (sent from the least significant bit first), a parity bit, and a stop bit (high level).

- Clock and Data Lines: The controller generates the clock signal and reads or writes data on the data line. The clock signal frequency is usually 250kHz, so the data transmission rate is 250kbps.

- Handshake Process: Before each data exchange, the host and controller perform a handshake. The host first sends a command byte to the controller, and the controller responds with one or more data bytes.
- Commands and Data: Common commands include request status (0x01) and read data (0x42). The controller's response data includes button status and analog joystick values.

USB Protocol:

USB is a universal serial bus standard used for device connection, communication, and power supply. In the USB protocol, data is serially transmitted over a pair of data lines (D+ and D-) using differential signals. Here are the basic principles of USB data reception:

- Data Encapsulation and Serial Transmission: The sending device encapsulates the data into a series of packets and sends these packets serially over the D+ and D- lines. The data packet format includes synchronization sequences, data payload, checksum, etc.
- Differential Reception: The receiving device receives data by measuring the voltage difference between the D+ and D- lines. The USB interface has strong resistance to power noise and external interference due to the use of differential signals.
- Packet Parsing and Checksum: The receiving device reassembles the received serial data into packets, then parses and checks them. If the packet checksum does not match the data payload, the receiving device can request the sending device to resend the data packet.